



# Tecnológico de Monterrey

**Campus Santa Fe**

**Database Normalization for the Project Supported by Generative AI Resources**

Software Construction and Decision Making

Estudiante:

Juan Pablo Briñez Corzo

Santiago Martin Vences

José Paulo Vélez Ortiz

Matrícula:

A01782050

A01787450

A01787207

@ 2026 OTHEA Games Studio

Fecha de entrega:

07 de abril

## **The importance of normalization**

Normalization is one of the fundamental aspects when designing an entity relationship diagram for a database. It allows us to avoid redundancy and improve the integrity of the data that is going to be stored. This is done through the three normal forms, rules to normalize databases:

- 1NF: A table must contain rows with unique and not null primary keys, its attributes must be atomic and identified by the primary key, and they mustn't have repeated values.
- 2NF: Comply with 1NF and eliminate all functional partial dependencies
- 3NF: Comply with previous NFs and eliminate transitive dependencies (pseudo primary keys)

Because of this, the database design may look more complex at first, but when doing this, the queries become simpler. This is because, through normalization, we can achieve simpler entities that represent the system as a whole, which helps to improve performance, scalability, and reduce modification problems to a minimum.

When programming, this can also be really useful, as when receiving data from a DB, if tables are normalized, it is easier to handle logic for applications. In general it is a good practice to normalize a database and it should be done when possible. There are cases where not normalizing it could be better, but following these standards will guarantee optimization and a better way to handle the DB.

## How generative AI tools can contribute to the improvement of ERD diagrams without replacing developers

Now, AI is a tool that can be really useful when normalizing a DB ERD, especially because it can do the task for you. However, AI is not a human, and it can make mistakes or add unnecessary things to the ERD. This happens because of the way AI tools are made; they work through statistics and complex mathematics that give a probable result. Because of this, most of the work done through AI must be reviewed by a human to avoid mistakes that can be prevented.

However, we have to say these tools make work far more efficient than before if used correctly. If prompted correctly, AI can work in parallel with humans to provide better results by offering advice on the current work being done. Because of this, AI tools shouldn't be demonized, but they also shouldn't replace developers, as developers are the ones who know how to handle information appropriately.

### Key Changes: Original Schema vs. Actual Schema

#### A. 1:1 Relationship in attributes

- Implemented the ***UNIQUE KEY player\_id (player\_id)*** constraint in the table ***attributes***. This converted a One-to-Many relationship into a 1:1 relationship. This guarantees that a single player profile can never accidentally possess duplicate sheets of attributes.

## B. Referential Integrity Fix

- The *death\_cause* column in runs evolved from a *VARCHAR(100)* into an *INT* mapped to an explicit Foreign Key: *CONSTRAINT fk\_run\_death\_enemy FOREIGN KEY (death\_cause) REFERENCES enemies (id) ON DELETE SET NULL*. This ensures run statistics are structurally bound to real cataloged enemies, preventing typos or phantom data.

## C. User Authentication Security

- Introduced *failed\_attempts INT DEFAULT 0* and *lock\_until DATETIME DEFAULT NULL* columns in users. This tracks malicious activity and locks out compromised accounts after repeated failed logins.

## D. Roguelite Persistence: *saved\_games* Table

Introduces a new table to manage progress:

```
CREATE TABLE `saved_games` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `user_id` int NOT NULL,  
  `slot_number` tinyint NOT NULL COMMENT '1..3',  
  `name` varchar(64) NOT NULL DEFAULT 'New Run',  
  `player_profile_id` int DEFAULT NULL,  
  `current_run_id` int DEFAULT NULL,  
  ...  
)
```

This handles the classic "Save Slot 1, 2, 3" mechanism using a *CHECK (slot\_number*

*between 1 and 3*) constraint and a composite **UNIQUE KEY** *uq\_user\_slot* (*user\_id*, *slot\_number*). It enables users to suspend and resume active gameplay by storing a pointer to the exact ongoing *current\_run\_id*.

Third Normal Form (3NF) — deliberate de-normalization

- **Rule:** The schema must be in 2NF, and there must be zero *transitive dependencies* (non-key columns whose values can be mathematically derived or predicted from other non-key columns).
- **Status:** Our Entity-Relationship model deliberately violates 3NF in some areas.

### 3. Justification for Violating 3NF

#### 1. Calculated Run Intervals (*runs.completion\_time\_seconds*)

- **Why it violates 3NF:** Total run duration is a transient data point that can be calculated dynamically at any time by executing a date subtraction function: *end\_time - start\_time*.
- **Justification:** In a roguelite, leaderboard sorting, global speedrun queries, and performance profiling are read-heavy actions. Calculating time differentials across millions of history rows via server-side CPU operations on every request would choke the database server. Storing the computed integer instantly optimizes high-frequency client reads.

#### 2. The Cache Layer Table (*player\_global\_stats*)

- **Why it violates 3NF:** Columns like *total\_runs*, *total\_victories*, or *total\_enemies\_defeated* introduce systemic redundancy. This data can already be

gathered by running massive aggregate *COUNT(\*)* or *SUM()* queries across your transactional tables (*runs* and *session\_enemies\_defeated*).

- **Justification:** Aggregating the lifetime telemetry of thousands of concurrent players in real time would crush game performance. The *player\_global\_stats* table acts as an **in-database persistence cache**. The game engine only needs to query a single row to instantly populate a player's profile menu, delegating historical writes to asynchronous backend threads or event-driven DB triggers.

### 3. Progressive Milestones (*player\_profiles.level* and *player\_profiles.total\_experience*)

- **Why it violates 3NF:** Because a character's level is determined strictly by an experience threshold curve, *level* exhibits a transitive dependency on *total\_experience*.
- **Justification:** When rendering UI element systems in the game client (such as character banners), the server must immediately stream a flat primitive integer. Forcing the engine to cross-reference an experience conversion lookup table during active game loop frames introduces unnecessary computational overhead.

**ERD:**

<https://drive.google.com/file/d/1eeIP72HPtKTR8DC3joNuS4ioLEAiWzBP/view?usp=sharing>